

System Description

Genesis Autonomous AI Agent

Version	1.0
Date	2026-05-06
Author	Jay Wingard
Subject System	Genesis Autonomous AI Agent
Classification	Public

Document History

Version / Date	Author	Description
1.0 (2026-05-06)	Jay Wingard	Initial release

Contents

1	System Description: Genesis Autonomous AI Agent	2
1.1	1. Executive Summary	2
1.2	2. System Architecture	2
1.2.1	2.1 Operational Context	2
1.2.2	2.2 Processing Model	3
1.2.3	2.3 Component Summary	3
1.3	3. Stakeholder Analysis	4
1.3.1	3.1 Human User	4
1.3.2	3.2 External Message Recipients	4
1.3.3	3.3 LLM Inference Providers	4
1.3.4	3.4 Hosting Infrastructure Provider	4
1.4	4. Autonomy Model and Decision-Making Controls	5
1.4.1	4.1 Earned Trust Framework	5
1.4.2	4.2 Action Classification	5
1.4.3	4.3 Approval Lifecycle	6
1.5	5. External Actions and Impact Surface	6
1.6	6. Data Handling	7
1.6.1	6.1 Categories of Stored Data	7
1.6.2	6.2 Data Flows to External Parties	7
1.6.3	6.3 Data Subject Considerations	7
1.7	7. Governance Controls	8
1.7.1	7.1 Enforcement Architecture	8
1.7.2	7.2 Health Monitoring	8
1.7.3	7.3 Audit Trail	9
1.7.4	7.4 Correction and Behavioral Adaptation	9
1.7.5	7.5 Controllability (ISO/IEC TS 8200:2024 Alignment)	9
1.8	8. Limitations and Identified Risks	10
1.8.1	8.1 Advisory-Layer Constraints	10
1.8.2	8.2 Third-Party LLM Dependency	10
1.8.3	8.3 Single-Operator Trust Assumption	10
1.8.4	8.4 Confabulation and Reasoning Reliability	10
1.9	9. Assessment Scope	11

System Description: Genesis Autonomous AI Agent

Version: 1.0 | Date: 2026-05-06 | Classification: Public

1. Executive Summary

Genesis is an autonomous AI agent system that operates as a persistent cognitive partner to a single human user. The system maintains continuous state across sessions, including long-term memory, behavioral drives, and an autonomy model that expands with demonstrated competence. Genesis can initiate actions independently of user input, including sending messages to external parties, executing code, and interacting with web services.

The risk surface presented by this type of system differs from those addressed by most existing governance frameworks. Classification systems, recommendation engines, and conversational chatbots operate within bounded interaction patterns where the AI responds to inputs and produces outputs within a defined scope. Genesis, by contrast, takes external actions with real-world consequences, modifies its own behavioral parameters through a learning pipeline, and operates proactively during periods without direct user oversight. These characteristics place it at the boundary of what current AI governance frameworks were designed to address, and identifying where those frameworks apply directly, where they require adaptation, and where genuine gaps exist is a central purpose of the assessments that follow.

This document describes the system's architecture, decision-making model, stakeholder relationships, data handling practices, and existing governance controls. It is written to provide the factual foundation that the subsequent NIST AI RMF profile (D1), ISO/IEC 42001 gap analysis (D2), and EU AI Act compliance assessment (D3) will reference.

2. System Architecture

2.1 Operational Context

Genesis runs as a containerized Linux application (Ubuntu 24.04) on cloud infrastructure. The runtime is approximately 94,000 lines of Python across 64 packages, backed by a SQLite database with write-ahead logging and a Qdrant vector database for semantic search. The system integrates with multiple external LLM providers for inference, primarily Anthropic Claude, with secondary routing to DeepInfra, Groq, and Mistral. A local Ollama instance handles embedding generation and lightweight classification tasks.

The system is single-tenant by design. One user, one instance, no shared state between deployments. Each deployment is operationally independent.

2.2 Processing Model

Genesis follows a perception-cognition-action loop that runs continuously, not only in response to user input.

During **perception**, the system collects signals from monitored sources including email, news feeds, web searches, and internal health metrics. These signals are triaged by relevance and urgency, then routed to appropriate processing pipelines. During **cognition**, incoming signals are evaluated against stored memory, current operational context, and four behavioral drives (Preservation, Curiosity, Cooperation, Competence). These drives function as sensitivity multipliers that bias attention and priority rather than serving as optimization targets. No single drive dominates unchecked. The architecture relies on tension between all four—each drive has a pathology when it runs unchecked, and health depends on their mutual constraint. During **action**, the system executes responses that range from purely internal operations (storing a memory, updating a classification score) to externally visible actions (sending a Telegram message, dispatching a background research session, pushing code to a repository).

The distinction between internal and external actions is governance-relevant. Internal actions are generally reversible and self-contained. External actions affect other people or systems and may be difficult or impossible to reverse, which is why they are subject to a separate classification and approval regime described in Section 4.

2.3 Component Summary

The system comprises several major subsystems, each with distinct governance implications:

The **Memory Store** provides persistent episodic and semantic memory, handling user interactions, synthesized knowledge, ingested reference material, and system-generated observations. The **LLM Router** manages model selection across providers, tracks cost and token usage, and implements circuit breaker patterns for provider resilience. The **Autonomy Manager** classifies actions by reversibility, enforces approval gates, and maintains trust evidence used for autonomy level calculations. The **Ego System** enables proactive decision-making through a two-role architecture that generates and evaluates proposals for autonomous action, subject to budget controls. The **Event Bus** provides structured logging and cross-component event distribution, capturing all significant system events with metadata. The **Outreach Pipeline** manages external message delivery across channels (Telegram, email) with scheduling, urgency classification, and salience scoring. The **Task Executor** handles multi-step autonomous work with progress tracking and verification gates. The **Guardian and Sentinel** subsystems provide health monitoring and active alarm response, respectively.

3. Stakeholder Analysis

3.1 Human User

The sole human operator functions simultaneously as the data controller, primary data subject, system administrator, and governance authority. This concentration of roles is unusual relative to most AI governance scenarios and simplifies several compliance considerations while introducing others.

The user sets behavioral constraints through steering rules (plain-text directives injected into the system's operational context), approves or rejects autonomy level promotions, resolves approval requests for actions classified as costly-reversible or irreversible, and retains the ability to pause, restart, or terminate the system at any time. The user owns all data produced by and stored within the system, with full direct access to the underlying database, file system, and vector store.

3.2 External Message Recipients

Individuals who receive communications initiated or composed by Genesis constitute an affected stakeholder group with limited visibility into the system. These recipients may not be aware that a message was AI-generated or AI-assisted. They have no direct relationship with the system, cannot modify its behavior, and are affected by the quality, timing, and appropriateness of the communications they receive.

The transparency implications for this group are assessed in the EU AI Act analysis (D3), particularly regarding Article 50 disclosure obligations for AI-generated content.

3.3 LLM Inference Providers

Third-party API services provide the language model inference that Genesis relies on for all reasoning, classification, and generation tasks. The primary provider is Anthropic (Claude models). Secondary providers (DeepInfra, Groq, Mistral) serve as routing targets for cost optimization and resilience. A locally hosted Ollama instance handles embedding generation.

These providers receive prompt content that may include user context, memory excerpts, and conversation history. Provider data retention and processing policies vary and are outside the system operator's direct control. This dependency is assessed under third-party risk in both the NIST assessment (GOV-6, third-party risk) and the ISO 42001 analysis (A.10, third-party and customer relationships).

3.4 Hosting Infrastructure Provider

The cloud VM provider hosting the Genesis container has physical access to the underlying disk, memory, and network traffic. The system mitigates this exposure through encrypted backups, secrets management (API keys stored in a permissions-restricted file outside the repository), and the general assumption that the hosting

provider operates under its own data processing agreements. Physical security of the hosting infrastructure is out of scope for this assessment.

4. Autonomy Model and Decision-Making Controls

4.1 Earned Trust Framework

Genesis operates under a graduated autonomy model in which the system's operational freedom expands as it accumulates evidence of appropriate behavior. The system does not begin with full autonomy. Trust is earned through a Bayesian evidence accumulation process.

The trust calculation uses a Beta distribution posterior with Laplace smoothing: the posterior probability is computed as $(\text{successes} + 1) / (\text{successes} + \text{corrections} + 2)$, where "successes" counts actions the user validated or did not correct, and "corrections" counts explicit user corrections or rejections. The posterior represents the system's estimated probability of taking an appropriate action within a given operational category.

Four operational levels are defined, each gated by a posterior threshold. Level 1 (the default, requiring no prior evidence) permits simple tool use such as reading files or checking system status. Level 2 (posterior ≥ 0.30) extends to known-pattern tasks such as running established procedures. Level 3 (posterior ≥ 0.50) permits novel task handling with a proposal checkpoint. Level 4 (posterior ≥ 0.70) enables proactive outreach, allowing the system to initiate communications without a direct user prompt.

Two design choices in this model are governance-relevant. First, the system cannot promote itself. Level advancement requires explicit user approval regardless of accumulated evidence. The Bayesian score indicates statistical readiness; the human decides whether to act on it. Second, the model includes an asymmetric regression mechanism: consecutive corrections regress the current operating level, while the historical maximum (earned level) is preserved as a record. Trust accumulates slowly and regresses quickly, creating a deliberate bias toward caution.

A known limitation: the Laplace smoothing that makes the formula well-defined at zero observations also means that small sample sizes produce high posteriors. With 5 successes and 1 correction, the posterior is 0.75—sufficient for L4 classification. Whether 6 observations constitute adequate evidence for the highest autonomy level is a design question that the formula does not answer. The manual promotion requirement partially mitigates this (the human can refuse to promote despite a qualifying posterior), but the formula itself does not distinguish between "0.75 based on 6 observations" and "0.75 based on 600 observations." The NIST assessment (D1, Appendix A) examines this limitation with production data.

4.2 Action Classification

Before execution, every action the system considers is classified into one of three reversibility categories: REVERSIBLE (undoable with no external impact), COSTLY_REVERSIBLE (undoable with some external impact), and IRREVERSIBLE (undoable with significant external impact).

(undoable but with effort, delay, or social cost), and IRREVERSIBLE (cannot be undone, including financial transactions and destructive operations).

This classification drives a hard gate in the execution path. Reversible actions proceed without approval at Level 2 and above. Costly-reversible actions require a proposal at Level 3. Irreversible actions always require explicit human approval, regardless of autonomy level, and the system will wait indefinitely for a response. There is no timeout that auto-approves an irreversible action. If the user never responds, the action never executes. This is the system's primary safety invariant.

4.3 Approval Lifecycle

When an action requires approval, the system creates a request record with full contextual metadata: what action is proposed, why it is proposed, what evidence supports the recommendation, and how the action is classified. The request is delivered to the user via Telegram with an inline keyboard for single-action or batch approval. Each resolution is attributed, recording whether approval came from a Telegram reply, a dashboard button click, or system-initiated cancellation (for example, when an alarm condition resolves before the user responds).

A consumed-at timestamp prevents double-execution of approved actions. Content-stable deduplication prevents the same logical request from being re-sent while a prior instance remains pending. A 24-hour staleness guard prevents accumulation of outdated approvals in the queue.

5. External Actions and Impact Surface

Genesis performs actions that affect systems and people beyond its own container. Understanding the scope and nature of these external actions is necessary for risk assessment.

In the domain of **communications**, the system can send messages via Telegram (direct messages, topic thread messages, inline keyboards), compose and send email via Gmail integration, and interact with web-based forms and forums through browser automation. All external communications at autonomy levels above L2 are subject to approval gate passage. The outreach pipeline applies delivery scheduling, salience scoring, and urgency classification before message dispatch.

In the domain of **code and infrastructure**, the system can execute shell commands within its container, read and modify files subject to protected path enforcement, spawn background sessions for autonomous work, and interact with Git repositories including commit, branch, and push operations. Push to the main branch is enforced as a hard block by a PreToolUse hook at the framework level.

In the domain of **web interaction**, the system can browse websites, fill forms, and interact with web services through browser automation. These interactions are bounded by browser session isolation, protected path enforcement on downloaded content, and approval gates for account-creating or form-submitting actions.

A hard constraint prohibits unsanctioned financial transactions. Every financial action requires individual, per-instance user approval. Prior authorization does

not carry forward to subsequent transactions. This constraint is enforced through steering rules and the action classification system.

6. Data Handling

6.1 Categories of Stored Data

The system stores several categories of data, each with different governance implications.

Episodic memory consists of records of user interactions and synthesized insights, retained indefinitely but subject to freshness scoring that reduces retrieval priority over time. **Knowledge units** are ingested reference materials, facts, and rules, also retained indefinitely. **Credentials** include API keys, account passwords, and authentication tokens, stored with restricted file permissions and encrypted in backups. Every credential access is logged to a dedicated audit table recording the accessor context, the accessed resource, and the timestamp. **Observations** are system-generated behavioral notes produced during reflection and learning cycles, aged out based on freshness. **Session transcripts** provide a complete record of all LLM interactions, tool calls, and outputs, retained indefinitely. **Activity logs** capture structured events from the event bus, and **approval records** preserve the full lifecycle of every approval request including resolution attribution.

6.2 Data Flows to External Parties

Prompt content transmitted to LLM providers may include memory excerpts, user context, and conversation history. The system does not control how providers handle this data after receipt. Outreach recipients receive composed messages that have passed through the approval pipeline. Code commits and pull request descriptions are transmitted to GitHub. Encrypted backups containing the full database, configuration, transcripts, and secrets are stored in a private GitHub repository.

6.3 Data Subject Considerations

The single-user deployment model simplifies data subject rights considerations. The user has full, direct access to all stored data through the database, vector store, and file system. Deletion of any data category is operationally straightforward. No data is held in systems the user cannot access or control.

External individuals who appear in the system's memory (contacts mentioned in research, outreach targets, people referenced in ingested content) represent a secondary data subject consideration. Their data exists within the user's private system with no public exposure, no shared access, and no automated decision-making about those individuals that produces legal or similarly significant effects as contemplated by GDPR Article 22.

7. Governance Controls

7.1 Enforcement Architecture

The system implements governance controls across multiple enforcement levels, arranged from strongest (architectural, not bypassable) to weakest (advisory, LLM-dependent).

At the **framework level**, PreToolUse hooks (`scripts/hooks/`) intercept file I/O operations before execution and enforce hard blocks (process exit code 2, not promptable) on modifications to critical paths defined in `config/protected_paths.yaml`. This enforcement operates at the Claude Code framework layer, below the LLM's control, and cannot be overridden by prompt content or system instructions.

At the **autonomy level**, the earned trust ceiling (`src/genesis/autonomy/state_machine.py`) gates action permission based on accumulated evidence and user-approved promotions. The system cannot exceed its current autonomy level regardless of the content of user instructions within a session.

At the **action classification level**, reversibility-based approval gates (`src/genesis/autonomy/src/genesis/autonomy/approval_gate.py`) create proposal checkpoints for costly-reversible actions and hard approval requirements for irreversible actions. The invariant that irreversible actions wait indefinitely for approval is enforced in the approval gate code (`timeout_seconds=None` for irreversible actions), not through LLM instruction.

At the **protected paths level**, a configuration file defines critical and sensitive file paths. Critical paths (including the relay infrastructure, autonomy protection code, and the protected paths configuration itself) cannot be modified from any relay or autonomous session. Sensitive paths require explicit approval plus self-review.

At the **steering rules level**, user-defined behavioral constraints are injected into the system's operational context. These operate at the advisory level, meaning the LLM reads and self-applies them. While this layer is not a security boundary on its own (a sufficiently adversarial prompt could theoretically cause the LLM to override advisory constraints), it functions within the defense-in-depth model where architectural enforcement layers catch what advisory layers may miss.

7.2 Health Monitoring

The Guardian subsystem performs periodic health checks measuring tick regularity, memory pressure, error spike rates, restart frequency, and pause state. When metric thresholds are crossed, the Guardian dispatches alerts.

The Sentinel operates as a second-tier active responder. When Guardian-detected alarms persist across multiple monitoring ticks (confirmed via 2-of-N debouncing to prevent single-tick false positives), the Sentinel classifies the alarm by tier and proposes investigation and remediation actions. These proposals are subject to the standard approval gate; Tier 1 alarms (defense failures, unwanted changes to critical paths) carry no timeout and wait indefinitely for human authorization.

7.3 Audit Trail

The system maintains overlapping audit surfaces. Approval records capture the full lifecycle of every approval request with resolution attribution. Session transcripts record all LLM interactions, tool calls, and outputs. The event bus logs all significant system events with structured metadata. Git history tracks all code changes with commit attribution.

These records are stored in the SQLite database and file system, both of which are directly accessible to the user for verification. The audit trail covers all system-internal actions but does not extend to actions taken by LLM providers on transmitted prompt content.

7.4 Correction and Behavioral Adaptation

When the user corrects a system action or decision, the correction is recorded as quantitative evidence (incrementing the corrections counter for the relevant operational category), potentially triggering Bayesian trust score regression and autonomy level reduction. The correction content is simultaneously stored as a negative training signal in the memory system, making it available for retrieval in future similar situations.

This mechanism creates domain-specific feedback loops: mistakes in one category make the system more cautious in that category, while success in unrelated categories does not compensate. The correction-to-regression path is intentionally asymmetric, requiring fewer corrections to regress than successes to advance, reflecting the design principle that the cost of unwarranted action exceeds the cost of unwarranted caution.

7.5 Controllability (ISO/IEC TS 8200:2024 Alignment)

ISO/IEC TS 8200:2024 defines controllability as a technical prerequisite for human oversight, not a property of the human operator. The standard specifies a two-condition test: (1) the system must represent its states to the controller, and (2) the system must accept and execute control instructions that cause state transitions.

Genesis satisfies both conditions. For state representation: the dashboard surfaces subsystem health, autonomy levels, pending approvals, event logs, and ego proposal status. Session transcripts and the event bus provide granular visibility into system behavior. The operator can query the database directly for any aspect of system state not surfaced through the interface.

For control instruction acceptance: the operator can pause or restart subsystems (state transition from active to paused), promote or regress autonomy levels (state transition across L1-L4), approve or reject proposals (state transition from pending to resolved), modify steering rules (behavioral state transition), and terminate the system entirely (state transition to stopped). These control instructions are accepted regardless of the system's current operational state—the kill switch, in particular, operates at the systemd layer and cannot be blocked by system behavior.

The controllability model breaks down in one scenario: when the operator is not

present to issue control instructions. TS 8200 assumes a controller who can observe states and issue instructions. During periods of proactive operation without human attention, the system's state is being represented (logged), but no controller is observing or issuing instructions. The Sentinel provides partial automated controllability (it can propose state transitions in response to health alarms), but these proposals still require human approval before execution.

8. Limitations and Identified Risks

8.1 Advisory-Layer Constraints

Steering rules and portions of the protected path enforcement rely on the LLM reading and self-applying constraints from its operational context. This advisory layer is not a security boundary. The system's defense-in-depth architecture mitigates this limitation, but users and assessors should understand that the advisory layer represents a behavioral expectation rather than a technical enforcement mechanism.

8.2 Third-Party LLM Dependency

All reasoning, classification, and generation capabilities depend on third-party LLM provider APIs. Provider outages degrade or halt system operation. Provider data handling practices are outside the operator's control. The circuit breaker pattern provides operational resilience through failover between providers, but does not address the fundamental architectural dependency on external inference services or the data governance implications of transmitting operational context to third parties.

8.3 Single-Operator Trust Assumption

The governance controls constrain the system's autonomy relative to its human operator. They do not constrain the operator's use of the system relative to the world. The system assumes a cooperative operator acting in good faith. There is no mechanism to prevent a malicious operator from directing the system to cause harm to third parties, because the system is designed to serve its operator's interests within the boundaries of its autonomy model.

8.4 Confabulation and Reasoning Reliability

LLM-based reasoning carries inherent confabulation risk. The system implements architectural mitigations including evidence requirements for claims, confidence labeling for uncertain assessments, and memory verification procedures. These reduce but do not eliminate the possibility of the system acting on fabricated or incorrect information. The risk is particularly relevant for actions that depend on recalled memory content or synthesized conclusions from multiple sources.

9. Assessment Scope

This system description covers a specific Genesis instance as deployed and operated by its single user. The subsequent framework assessments (NIST AI RMF, ISO/IEC 42001, EU AI Act) evaluate this specific deployed system with its specific governance controls, not the Genesis codebase as a generic software product or theoretical architecture.

The following items are explicitly out of scope: other deployments or forks of the open-source Genesis codebase; the underlying LLM models themselves (which are subject to separate provider-level assessments and, in the case of Claude, to GPAI obligations under EU AI Act Chapter V); the physical security of the hosting infrastructure; and the user's broader IT environment beyond the Genesis container boundary.